

Inferring the Test from the Constraints Alone

Strip away the programme, the country, and the document. What's left is a set of constraints, and constraints determine design. Let me reason only from those.

1. The constraints, and what they force

Constraint	Design consequence
45 questions in 45 minutes	~1 minute/question — but the <i>questions</i> won't all take a minute. Some will take 20 s, some 3 min. The average is only forced by the total.
Three subjects	Triage — the committee expects candidates to be uneven across them.
85% cutoff	You may lose at most 6–7 marks . That's not a test of knowledge; that's a test of near-perfection under pressure.
"Screen out as much as possible"	The test is a discriminator, not a curriculum sample . It optimises for variance at the top.

The last two are the master constraints. Everything else follows.

2. What an 85% cutoff actually implies

A cutoff only means something if the *distribution of scores* clusters tightly around it. If the median candidate scores 90%, the cutoff is decorative. If the median scores 55%, the cutoff is a wall.

So the committee has a target distribution in mind: **most candidates land below 85, a small cluster lands exactly around it, and a thin tail clears it comfortably**. The test is engineered so that a candidate who is "good" in the ordinary sense (comfortable on a WAEC paper, say) lands around 70–78. Only the genuinely fluent — fast, broad, precise readers and reasoners — reach 88+.

This tells us the questions are **not calibrated to reward competence**. They're calibrated to expose *specific* weaknesses.

3. The three filters they'll use

Any question can filter on three axes. A committee trying to eliminate aggressively will mix all three.

(a) Time as a covert filter. Not by making the test short — by making individual questions long enough that a candidate *cannot* finish by working carefully. The candidate must decide, mid-question, whether to push or abandon. This tests judgment and speed, not knowledge. Expect 6–10 questions that are technically solvable in 3 minutes and effectively unsolvable in 1.

(b) Traps on the "obvious" answer. Each question will have one distractor that is correct under a careless reading. A question about the author's *tone* will have a distractor for the author's *subject*. A probability question will have a distractor for the complement. A GK question will have a distractor that's true in a neighbouring context. The candidate who scans loses 3 marks they thought they'd won.

(c) Breadth without a syllabus. If the committee published a topic list, coaching would collapse the discriminative power. So the topics are wide, unpredictable, and include "adjacent" knowledge that a well-read person absorbs and a crammer doesn't.

4. What each subject will actually look like

English (~15 questions)

Not synonyms. Not "choose the correct form." Those have high floors and are coachable in a weekend.

Instead: **dense passage inference**. Expect 2–3 passages of 100–150 words — academic, abstract, or argumentative — with 2–3 questions each. The questions will ask about **attitude, function, and implication**, not fact. Distractors will be *approximately* right: a sentence that's true of the passage but answers a different question. One or two word-in-context questions where the most common meaning is the trap.

A candidate who reads quickly and carefully gains 3–4 marks here. A candidate who reads quickly and carelessly loses the same. That's a 6–8 mark swing on English alone — enough to move a borderline candidate from 87% to 79%.

Maths (~15 questions)

Not computation. Computation is a time sink, not a filter. Expect **multi-step word problems and identities**:

- Algebraic manipulation (e.g. $x + 1/x = k \rightarrow x^2 + 1/x^2 = 2$ logical hops)
- Probability with "at least" or "without replacement" — the double-counting trap
- Ratio and proportion disguised as a word problem
- Sequences where the rule isn't linear
- A small data-interpretation or geometry question where the *setup* hides the algebra

Every question answerable by a strong candidate in 90 s, unanswerable in 60 s by a merely competent one. The filter isn't whether you *can* solve it — it's whether you can solve 15 of them in 15 minutes.

General Knowledge (~15 questions)

This is where the committee hides the sharpest filter, because GK is the least coachable and most correlated with "broadly educated, curious, well-read."

Not current affairs (changes yearly, unfair, unverifiable). Instead: **foundational knowledge across a wide domain** — basic genetics, basic economics, constitutional structure, international institutions, major historical events, geography of one's own region. Questions are binary: either you know it or you don't, and no amount of reasoning substitutes for having read widely.

Expect 2–3 questions that look like general knowledge but actually test *reasoning* — spotting the assumption in a short argument, identifying a logical error, interpreting a small statistic. These are the "silent killers" because they can't be studied for; they reward how someone thinks.

5. The overall texture of the test

Here's my prediction of what a candidate *feels*:

- **Questions 1–10 feel easy** — deceptively. Two or three of them are traps. The candidate's confidence rises and their vigilance drops.
- **Questions 11–30 feel "normal"** — this is where most candidates are actually failing without knowing it. Each question is 1 step harder than it should be.
- **Questions 31–45 feel hard** — some are unsolvable in the time remaining. The candidate must guess or abandon.

Subjects are likely **interleaved**, not blocked. Blocked subjects let a candidate warm up; interleaving forces a context switch on every question. That's an executive-function tax on top of everything else.

No calculator. No formula sheet. No partial credit. Every tool that would neutralise the time pressure is removed.

6. What the failure modes are engineered to catch

Failure mode	How it's caught
Speed without care	Trap distractors that reward careful reading
Knowledge without reasoning	Multi-step questions where each step is trivial but the chain isn't
Reasoning without breadth	GK questions that can't be deduced
Breadth without depth	Passage inference that requires sustained attention
Panic under pressure	Long questions late in the paper, with time already short
Overconfidence	Easy-looking questions that hide a qualifier

The test isn't testing whether you know things. It's testing whether you can **decide well, under load, on unfamiliar material, quickly, and without making careless mistakes**. That's what an 85% cutoff on 45 minutes actually measures.

7. The bottom line

If I had to bet on what the questions look like: **dense, precise, and designed to punish the second-best reading of them**. Not obscure — *adversarial*. The correct answer is usually reachable, but there's always a plausible wrong answer sitting right next to it, waiting for the candidate who is in a hurry. The test doesn't ask "do you know this?" It asks "can you find the right answer *and* avoid the wrong one, in under a minute, 45 times in a row?"

Very few people can. That's the point.

THE MASTER BRIEF

A single, complete instruction for building the DCF Aptitude Prep app — written so a non-technical person can hand it to anyone and get the right thing built.

Introduction — What we're building and why

We are building a **test-preparation app** for a scholarship aptitude exam called the Dreamers College Fund (DCF). The app will:

1. Let a candidate sit a **full simulated exam** — 60 questions across three subjects, timed, scored, with a pass/fail verdict.
2. Let a candidate **drill a specific English skill** — for example, "semicolon use" or "reading comprehension" — with instant feedback.
3. **Work offline** on an Android phone, look like a real app, and install to the home screen.
4. **Track progress over time** — every attempt, every score, every weak spot — so the candidate knows exactly where to improve.
5. **Never repeat a question** until the entire pool is exhausted, so every attempt feels fresh.

The rest of this brief explains **what we're building**, **why each decision was made**, and **exactly how to get it online** — in plain English, without jargon.

PART 1 — The Big Picture

Think of the app as **three layers stacked on top of each other**:

- **The top layer** is what the candidate sees and touches: buttons, screens, a countdown timer, coloured answer tiles, a score ring.
- **The middle layer** is the logic: how questions are chosen, how time is counted, how scores are calculated, how progress is remembered.
- **The bottom layer** is the plumbing that makes it installable and work without internet: two small files (a "manifest" and a "service worker") that tell the phone "this is an app" and "save these files so you can use them offline."

We build these in that order. First the visible thing, then how it behaves, then how it survives without WiFi.

PART 2 — The Test Itself

Structure

The full exam is **60 questions** to be completed in **45 minutes**. This is a *whole-test* timer, not a per-question timer. When the clock hits zero, the app submits automatically. That single design choice is deliberate: it forces the candidate to *choose* how long to spend on each question, which is exactly what the real exam will do.

Three difficulty tiers, with weighted points

Every question belongs to one of three tiers:

Tier	Points per question	Count	Total
Easy	5	20	100
Hard	10	20	200
Difficult	15	20	300
Total		60	600

The candidate starts with Easy and works up. Harder questions are worth more, which means **the top of the score range is where the competition really happens**. Getting all Easy right earns only 100 points — you cannot pass on easy questions alone.

The cutoff

The pass mark is **510 out of 600** — **that's 85%**. This is deliberately high. A candidate may lose **at most 90 points**, which is 6 Easy questions, or 3 Difficult ones, or a mix. The test is designed to reward near-perfection, not competence.

Subject mix

Three subjects are tested:

- **English** — reading, vocabulary in context, grammar (subject–verb agreement, tense, pronoun reference, semicolons).
- **Mathematics** — word problems, algebra, probability, sequences, geometry. **No calculator allowed.**
- **General Knowledge** — genetics, government, economics, science, geography. Deliberately **not current affairs**, because current affairs change yearly and are unfair to test on.

Question design principles — and the traps they hide

Every question in the bank must follow these rules:

1. **No calculator required.** All arithmetic is mental-friendly (nice fractions, clean numbers).
2. **One plausible trap.** Each question has one distractor that is correct under a careless reading. This punishes speed without care.
3. **Context over recall.** Vocabulary questions use words inside sentences, not "what does X mean?" A candidate must reason from clues.
4. **Longer reading passages.** English comprehension uses passages of ~100–150 words with 2–3 questions each. Short passages are too easy to skim.
5. **Calibrated difficulty.** Easy $\approx$ IGCSE Core. Hard $\approx$ SAT Level 1 / WAEC Advanced. Difficult $\approx$ SAT Level 2 / Further Maths.
6. **Fair but adversarial.** The correct answer is reachable, but the wrong answer is right there next to it, waiting for someone in a hurry.

Non-repetition

The app remembers every question ever served. On the next attempt, it draws from the *unused* pool first. When a subject-tier cell runs out of fresh questions, it starts reusing old ones — but shuffles their answer options so the layout feels new.

PART 3 — The English Drill Mode

The full test is a screening tool. It tells you *that* you failed. It doesn't tell you *why*. The Drill Mode fills that gap.

What it is

A separate, isolated practice session with:

- **20 questions**, drawn from one or more skills
- **15 minutes**, not 45
- **Immediate feedback** — right or wrong is revealed after every answer, not at the end
- **No going back** — once you answer, you move on
- **Skill-level tracking** — the app remembers your accuracy per skill across every drill you've ever done

The six skills it tracks

1. **Vocabulary in Context** — inferring meaning from sentence clues
2. **Reading Comprehension** — tone, purpose, function of specific phrases
3. **Subject–Verb Agreement** — singular/plural matching
4. **Tense Consistency** — correct verb tense in context
5. **Pronoun Reference** — clear and correct pronoun use
6. **Semicolons** — joining independent clauses correctly

Why it matters

After three or four drills, the app has built a **lifetime accuracy bar** for each skill. It will show you, for example: "*Semicolons: 82%. Pronoun Reference: 45%.*" You then know exactly where to spend your study time. This turns a test-prep app into a diagnostic tool.

PART 4 — The App Screens

The candidate moves through a series of screens. Each screen shows only one thing at a time. (We had a bug where multiple screens tried to show at once — the fix is to give screens a very high-priority visibility rule so only the active one can appear.)

The screens in order:

1. **Home** — logo, format summary, quick-start buttons, progress stats.
2. **Instructions** — plain-language overview of the test.
3. **Subject Picker** — tap one, two, or all three subjects. The app splits 60 questions evenly across your choice.
4. **Test Session** — question, timer, options, prev/next.

5. **Results** — big score ring, pass/fail, subject breakdown, difficulty breakdown, personalised recommendations.
6. **Review** — every question with your answer and the correct answer, colour-coded.
7. **Drill Picker** — tap the skills you want to practice.
8. **Drill Session** — like the test, but with instant feedback.
9. **Drill Results** — session summary plus lifetime mastery bars per skill.

The recommendation engine

After every test, the app generates a personalised set of study recommendations based on your performance. If your Maths is below 85%, it tells you to focus on identities, probability without replacement, and quadratic roots. If your English is weak, it points you to reading comprehension and grammar. If your Easy questions failed more than your Hard ones, it flags **fundamental gaps** — because that pattern means you're not losing to test technique, you're losing to content.

PART 5 — The Technical Build (explained simply)

The app is **four files in one folder**. Nothing else is needed.

File	What it is (plain English)
<code>index.html</code>	The app itself — everything visible is defined here, plus the logic in a hidden script at the bottom
<code>manifest.json</code>	An "ID card" that tells the phone "this is an installable app, here's its name, here's its icon, here's its colour"
<code>sw.js</code>	A "service worker" — a small helper that saves the app files to the phone so they work without internet
<code>icon.svg</code> (and 2 variants)	The app icon — a small picture file that renders at any size

Why a single HTML file? Because it means there is only one thing to keep track of. All the styling, all the logic, all the content lives in one document. There is no build step, no compiler, no dependency manager. You open the file, it runs.

How does it work offline? When a user opens the app the first time, the service worker *caches* — that is, saves — every file the app needs. From then on, the phone reads from that saved copy. No internet is required. This is why the app can run in airplane mode.

How does it install to the home screen? The manifest tells the browser "this can be installed." Chrome on Android then offers an **"Install app"** option in its menu. When the user taps it, the app gets its own home-screen icon, opens full-screen (no browser bar), and behaves exactly like a native app.

How does it remember your progress? Using something called `localStorage` — a small memory on your phone that only this app can read. It stores your attempt history, your drill statistics, and the list of questions you've already seen. Nothing is sent to a server. Everything stays on your device.

PART 6 — The Icon & Branding

The icon is called **"The Scholar's Star."** It's a graduation cap with two golden sparkles above it, set on a purple-to-pink gradient background.

Why this design

- **Renders cleanly at any size.** The cap shape is bold enough to read at 48 pixels (a phone launcher size) and elegant enough for a full-screen splash.
- **Uses a warm gold accent** to signal achievement — the same visual language used by Duolingo and Khan Academy.
- **Avoids the obvious.** No books, no pencils, no owls. The cap is iconic without being clichéd.

Three variants

1. **Primary icon** (`icon.svg`) — what the phone shows on the home screen.
2. **Maskable icon** (`icon-maskable.svg`) — same design but shrunk to 72% centred, so nothing gets clipped by Android's various adaptive shapes (circle, squircle, teardrop, etc.).
3. **Monogram** (`icon-monogram.svg`) — a simplified "D" for the browser tab, notification badges, and Android themed icons.

The manifest declares all three, with the correct "purpose" tag on each: `any`, `maskable`, `monochrome`.

PART 7 — Getting It Online (Deployment)

We use **GitHub Pages** — a free service that turns a folder of files into a live website. This is what makes the app reachable by any phone, anywhere.

Why GitHub Pages

- **Free.** No hosting cost.
- **HTTPS automatically.** The service worker (offline mode) only works on secure connections — GitHub Pages provides that for free.
- **Simple.** You upload files once, and the site is live at a public link.

Step-by-step (non-technical)

1. **Create a GitHub account** — go to github.com and sign up. It's free.
2. **Create a new repository** — think of a repository as a folder on the internet. Give it a name like `dcf-prep`.
3. **Upload the four files** — drag `index.html`, `manifest.json`, `sw.js`, and the icon files into the repository's upload area. Commit.
4. **Enable GitHub Pages** — in the repository settings, find "Pages" in the sidebar. Choose branch `main`, folder `/ (root)`. Save.
5. **Wait 60 seconds.** GitHub will show you a link like `https://your-username.github.io/dcf-prep/`. That is your app.
6. **Open the link in Chrome on Android.** Tap the three-dot menu, tap "Install app." Done.

The update workflow

Whenever you change the app — new questions, new features — do this:

1. **Bump the cache version** inside `sw.js`. This is the single most important step. The service worker checks its version number; if it's unchanged, it serves the old files even if you've uploaded new ones. Change `v7` to `v8` and the app will refresh on next launch.
2. **Upload the changed files** to GitHub (drag and drop works).
3. **On the phone**, force-close the app and reopen it. The new version installs quietly in the background.

PART 8 — The Study Plan (for the candidate, not the app)

The app is a tool. Here is how to use it.

Week-by-week

Week	Focus	What to do
1	Vocabulary in Context	15 drill questions daily. Read one editorial column and note 5 words in context.
2	SAT Reading: Tone & Purpose	2 passages daily. For each, write one sentence on tone, one on purpose.
3	Grammar: Subject-Verb Agreement	20 drill questions daily. Focus on the "ignore the prepositional phrase" rule.
4	Grammar: Tense + Pronouns + Semicolons	20 mixed drill questions daily.
5+	Full test simulations	One full 60-question test per week. Use drill mode between tests.

The three habits that matter

1. **Read every day.** Even 10 minutes of long-form prose builds reading stamina. The SAT doesn't test vocabulary; it tests whether you can hold a paragraph in your head.
2. **Do timed drills.** Untimed practice builds knowledge. Timed practice builds *judgment*, which is what the real test measures.
3. **Review every wrong answer.** Reading the answer key and moving on is useless. The question is not "*what's the right answer?*" — it's "*why did the wrong answer look right to me?*" That is the only question that changes your score.

PART 9 — What the Real Test Will Try to Do to You

This is the strategic layer. Based on the constraints alone (45 questions in 45 minutes, 85% cutoff, three subjects), here's what the real DCF test is designed to do:

- **Use time as a covert filter.** Not by making the test short, but by making some questions long enough that you can't finish by working carefully. You must learn to abandon.
- **Trap the careless reader.** Every question has one distractor that's correct under a careless reading. The correct answer is not hard — it's just harder to find when you're rushing.

- **Interleave the subjects.** Not blocked — mixed. Because blocking lets a candidate warm up. Interleaving forces a context switch on every question, adding executive-load on top of the difficulty.
- **Test reasoning, not recall.** Especially in General Knowledge. A question that *looks* like a fact question may actually be asking you to spot the assumption in a short argument.

The three failure modes the test is engineered to catch:

1. Speed without care — losing points to trap answers.
2. Knowledge without reasoning — knowing facts but not applying them.
3. Reasoning without breadth — smart but not well-read.

The test isn't asking "do you know things?" It's asking "can you decide well, under load, on unfamiliar material, quickly, and without making careless mistakes — 45 times in a row?"

Very few people can. That is the point.

PART 10 — Iteration & Improvement

The app is not finished when it's built. It's finished when it stops helping. So the plan is:

- **Every attempt, note which subject and tier you lost the most points on.**
- **Use drill mode between tests** to close the biggest gap.
- **When the question pool runs dry** on a specific subject-tier, add more questions at that exact calibration.
- **When a question feels mis-tiered** — when an "Easy" question in your bank takes 3 minutes, or a "Difficult" question takes 30 seconds — retag it. Calibration drifts over time and needs maintenance.
- **When the icon feels stale**, redesign it. Brands are living things.

PART 11 — Summary of Every Deliverable

#	Deliverable	Purpose
1	index.html	The whole app — screens, logic, question bank, styling.
2	manifest.json	Makes the app installable on Android.
3	sw.js	Makes the app work offline. Version-bump on every release.
4	icon.svg	Primary launcher icon.
5	icon-maskable.svg	Safe-zone variant for Android adaptive shapes.
6	icon-monogram.svg	Simplified variant for tabs and notification badges.
7	GitHub Pages deployment	Live HTTPS URL to share and install from.
8	Study plan	The human habit layer that makes the app effective.
9	Strategic analysis	What the real exam will do, so the app simulates it honestly.

The One Rule That Governs Everything Else

Every decision in this brief is downstream of one constraint: the exam is designed to screen people out, not to teach them. So the app must do two contradictory things at once — it must be **hard enough to be honest** (so the candidate isn't surprised on test day), and it must be **specific enough to be useful** (so the candidate learns *why* they're failing, not just *that* they are).

That contradiction is the whole design. Everything else is implementation.

This is the complete brief. You can hand it to a developer, paste it into an AI, or use it as your own roadmap. It explains not just *what* to build, but *why* each piece exists — and where the traps are.

Claude

That combination of numbers tells you a lot on its own, actually — let me reason through it without touching the uploaded document.

The core tension: 45Q/45min + 85% cutoff = a discriminator, not a knowledge test

One minute per question sounds generous until you realize an 85% cutoff means the exam has to still function as a filter even after most applicants have "studied hard." If the whole paper were uniformly WAEC-level, too many prepared candidates would clear 85% and the test wouldn't screen anyone out — defeating its purpose. If it were uniformly brutal, almost nobody would

reach 85% and the cutoff would be meaningless noise. Neither extreme is useful to a committee that explicitly wants to screen out as many people as possible from a large pool.

The only design that makes an 85% cutoff *meaningful* is a **bimodal difficulty curve**: the large majority of questions (I'd guess 35–40 of the 45) are fair, curriculum-appropriate, and gettable by anyone genuinely prepared — these exist so the exam isn't arbitrary. A smaller cluster (5–10 questions) are deliberately engineered to be missed by all but the sharpest candidates. Those few questions do almost all the actual "screening" work — everything else just filters out the unprepared. This is standard practice in any high-stakes gatekeeping exam (JAMB, SHL-style corporate aptitude tests, bar exams): most of the paper confirms competence, a small tail measures ceiling.

What that implies about question *character*, not just difficulty:

Given 60 seconds/question on average, the hard tail can't rely on length or heavy computation — there's no time for that. So the difficulty has to be *hidden inside apparent simplicity* rather than announced by complexity. That points toward specific tactics:

- **Trap-based precision, not raw difficulty.** Questions engineered to be answered wrong quickly rather than answered right slowly — a sign flip, an "except/NOT" qualifier buried in the stem, a distractor that's correct for a *slightly* different question than the one asked.
- **Pattern recognition over brute force.** A question that's trivial if you spot the shortcut (e.g., a difference-of-squares trick, a symmetry in a word problem) and painfully slow if you don't — this cleanly separates fluency from memorization without needing exotic content.
- **Extraneous or reordered information.** Word problems with one irrelevant number, or data presented in an order that punishes skimming — testing careful reading under pressure rather than knowledge itself.
- **No signal about which questions are the "real" filters.** If the hard ones were identifiable by position or framing, strong candidates would just budget time around them. A well-designed screening exam disguises its discriminators — a "hard" one might read shorter and simpler than an "easy" one.

Per subject, in the abstract:

- **English** — given "communication skill" is explicitly a trait they care about, I'd expect the filtering to sit in *precision under ambiguity*: which of four nearly-identical sentences is the only grammatically clean one; a word's meaning shifting with register/connotation rather than dictionary definition; a comprehension question with a "which is NOT stated" trap. Vocabulary tested by consequence of use, not recall of definition.
- **Mathematics** — I'd expect most items to be direct WAEC/JAMB-style application (no trap, just competence), with the filtering tail hidden in things like: a problem solvable in 10 seconds via a pattern versus 90 seconds via brute algebra; a quadratic/absolute-value item where the "obvious" single answer is wrong because there are two valid solutions; a problem with a plausible-but-irrelevant number inserted specifically to catch someone pattern-matching instead of reading.
- **General Knowledge** — this is likely the subject with the *least* prepared-for content on purpose. If Math and English can be studied into submission, GK is the section where "genuine breadth of curiosity" (which DreamRite names as a trait they want) actually shows up — meaning a chunk of it may be near-impossible for the average candidate regardless of prep, functioning less as a knowledge check and more as a tie-breaker among people who already did well elsewhere.

One honest caveat: this is inference from incentive structure, not confirmed knowledge of DreamRite's actual paper — I'm reasoning from "what would a rational committee do with these constraints," not from anything I've read of theirs. If you want, I can now go build a practice tier explicitly modeled on this trap/precision logic — short-looking, fast-to-misread questions rather than long, obviously-hard ones — which is a meaningfully different training target than "make it 5x harder."

Building a DCF Scholarship Practice Exam — The Complete Story, Step by Step

This document is written as if you're explaining the whole project to someone who has never touched code and doesn't know what "React" or "localStorage" means. It walks through everything that was asked for, in the order it happened, and — more importantly — *why* each decision was made the way it was. By the end, you should be able to hand this document to anyone (a developer, another AI, a friend) and they'd understand exactly what was built and the reasoning behind every choice.

0. The Starting Point: What Are We Actually Trying to Do?

The Dreamers College Fund (DCF) scholarship has a multi-stage selection process. One of those stages is a computer-based aptitude test. The goal of this whole project was simple to state but hard to execute well: **build a practice tool that prepares a candidate for that real test as faithfully as possible** — not just in content, but in *feel*, *pressure*, and *psychology*.

That single goal branched into a long list of sub-problems, because "prepare someone for a real exam" turns out to require thinking about far more than just writing quiz questions. Below is every piece of that puzzle, in the order it was tackled.

Phase 1 — Start With a Paper-Based Practice Set

The ask: Before any app existed, the first thing built was a plain, printable practice test — 120 multiple-choice questions covering five subjects (English, Mathematics, Economics, Biology, and General Knowledge), based on a scholarship guide document that had been shared.

The strategy:

- Split the 120 questions evenly-ish across the five subjects the guide mentioned.
- Wrote them at a level appropriate to the exam's stated audience (WAEC/JAMB-level Nigerian secondary school students).
- Included an answer key and a simple scoring guide at the end, so it could be used without any digital tool at all — just print it and take it with a pen.
- Delivered it first as a Markdown (plain text) document, then converted into a proper PDF using a document-conversion tool, so it would look clean and professional when printed or shared.

Why start here? Because before building anything complicated, it's worth getting the *content* right on the simplest possible format. A PDF can't have bugs. It was a safe, fast way to validate "does this look like a real practice test" before investing in an interactive version.

Phase 2 — The Real Rules Change

The ask: New information arrived about the *actual* test structure. This wasn't a guess anymore — it was the real rulebook:

- Only three subjects count: English, Mathematics, and General Knowledge (Economics and Biology were dropped).
- The test is scored from 0 to 600 points.
- You need at least 510 points (85%) to pass — a notably high bar.
- Questions come in three difficulty bands: **Easy** (worth 5 points each), **Hard** (worth 10 points each), and **Difficult** (worth 15 points each).
- Everyone starts on Easy questions.

The strategy: This information was saved as a standing fact about the project (so it wouldn't be forgotten or contradicted later), and every future version of the practice tool was rebuilt to strictly follow these rules — subject list, point values, and pass mark all had to match exactly, because the whole point of a practice tool is that it mirrors the real thing.

Why this mattered: A practice test that scores differently from the real test teaches the wrong instincts. If practicing on a 100-point scale makes 70 feel "good enough," but the real test needs 85%, the practice tool would be quietly setting someone up to underestimate the bar. Precision here isn't a nitpick — it's the whole value of practicing.

Phase 3 — From Paper to an Interactive, Adaptive Quiz App

The ask: Turn the static question set into an actual computer-based test (CBT) application — one that behaves like the real online exam rather than a piece of paper.

The strategy — and the key design decision: The real exam's rule "you start on Easy" implied something important: the test is probably **adaptive**, meaning your performance decides what you see next, not just what order questions come in. So the app was built so that:

- You always start on Easy questions for a subject.
- If you score well enough on Easy (a 50% threshold was chosen, since the actual advancement rule wasn't specified), you "unlock" Hard questions for that subject.
- Doing well on Hard unlocks Difficult.
- If you don't clear the bar, you stop there for that subject, keep whatever points you earned, and move to the next subject.
- This repeats for all three subjects, and your points add up toward the 0–600 total.

Why adaptive, and why explain the assumption? Because when a rule isn't fully specified (the exact "how good do I have to be to advance" percentage wasn't given), the honest thing to do is *pick a sensible number and say so out loud* — rather than silently guessing and letting the user think it's an official rule. This shows up throughout the project: whenever an assumption had to be made, it was stated plainly on the test's own start screen, not hidden in the code.

A technical idea worth explaining in plain terms: The app was built using a technology called "AI-generated content via API calls" at this stage — meaning each time you took the test, it would ask an AI (in real time, over the internet) to invent a fresh batch of questions for whatever subject and difficulty you were on. This gave genuinely unlimited variety. It had one big weakness, though, which shows up in the next phase.

Phase 4 — No Calculators Allowed

The ask: The Mathematics questions shouldn't require a calculator to solve.

The strategy: Every math question — however it's generated — needs to resolve to "clean" numbers: whole-number answers, percentages that come out even, fractions that simplify neatly, and so on. This meant going through every math question type and deliberately engineering the numbers involved (for example, only using percentages like 10%, 20%, 25%, 50% applied to numbers specifically chosen to divide evenly) so that mental arithmetic or simple pencil-and-paper working is always enough. Ugly decimals, unresolvable divisions, and messy square roots were avoided by design, not by accident.

Phase 5 — It Has to Work Completely Offline

The ask: The app should function with no internet connection at all.

The strategy — and why Phase 3's approach had to be undone: Remember the "ask an AI in real time for new questions" approach from Phase 3? That requires an internet connection every single time, which directly conflicts with "works offline." So that entire approach was replaced with two offline-friendly alternatives:

1. **For Mathematics:** Instead of asking an AI for questions, the app itself was given a set of *question-writing recipes* (templates) with random numbers plugged in each time. For example, one recipe always builds a percentage question, another always builds a "solve for x" equation, another builds a logic puzzle about who's taller than whom. Because the app *is* the question-writer now, no internet call is needed — it can generate infinite fresh variations entirely on the device, instantly.
2. **For English and General Knowledge:** These are much harder to generate well by pure formula (a computer can't easily invent a good, grammatically clean vocabulary question the way it can invent a math problem). So instead, a large hand-written bank of quality questions was built for these subjects, and the app randomly shuffles and samples from that bank each time, so it doesn't always show the same order or feel repetitive.

A quality-control habit introduced here (and used constantly afterward): Because the math questions are now built from formulas rather than checked by an AI in the moment, there's a real risk of the formula quietly producing a *wrong* answer sometimes (bad math, or a formula that occasionally breaks). So every single math "recipe" was tested separately, outside the app, by running it thousands of times with random numbers and checking that the answer it produces is always actually correct. This testing habit caught several real bugs along the way — including, much later, an actual infinite-loop bug where one formula could get stuck forever under a rare unlucky combination of random numbers. Testing formulas before trusting them became a non-negotiable step for every future round of math questions.

Phase 6 — More Variety, Real Difficulty, and a Ticking Clock

The ask (three things at once): Add more question variety, make each difficulty tier noticeably harder, and add a countdown timer for each question that locks you out once time runs out.

The strategy for variety: More "recipes" were added for math (more question types per difficulty level), and the hand-written English/General Knowledge banks were expanded with more items, so repeats would be rarer.

The strategy for difficulty: Since "3x harder" isn't a number you can literally calculate, it was treated as a *direction*, not a formula — each tier was rebuilt with genuinely more demanding content:

- Easy went from single-step sums to two-step problems.
- Hard introduced things like solving two equations at once, or working out how long two people take to finish a job together.
- Difficult introduced multi-step logic puzzles, reversing percentages ("the price went up 20% and is now ₦240 — what was it before?"), and problems requiring two separate pieces of information to be combined before you could even start solving.

The strategy for the timer: Each difficulty tier got its own time limit per question (Easy questions got less time, Difficult questions got more, since they need more thinking). Visually, a shrinking colored bar and a countdown number were added, turning red and pulsing in the final seconds — mimicking the real anxiety of exam time pressure. When time hit zero, the question would lock immediately: no more answering, the correct answer gets revealed, and you move on — exactly like a real hard-cutoff exam clock.

Phase 7 — Recognizing the Exam's "Personality" and Matching It

The ask: A moment of stepping back happened here — noticing that the whole structure (adaptive tiers, timed, multiple-choice) resembles the SAT, and asking: shouldn't the *content* match that pedigree too? Alongside this came two more requests: broaden General Knowledge beyond just current-affairs/government trivia, and increase the total number of questions to 60.

The strategy:

- **Mathematics and English were rebuilt to genuinely resemble SAT-style questions** — not just harder arithmetic, but the actual *format* of SAT-style items: functions written as $f(x)$, quadratic equations, coordinate geometry, grammar "spot the error" questions, vocabulary used in the way SAT/GRE-style tests use it, sentence-completion questions with a logical twist ("although his argument was ____, the jury remained ____").
- **General Knowledge was deliberately rebalanced** away from being mostly Nigerian government/current-affairs trivia, and toward science, technology, history, geography, and culture — because "general knowledge" should mean broad curiosity, not memorized civics facts.
- **The question count was raised to 60** (from an original 54), which meant recalculating how many questions belong in each difficulty tier per subject so the numbers still divided evenly and the scoring still worked out to the official 0–600 scale.

An important habit reinforced here: asking before guessing. Because this round of changes involved several genuinely ambiguous decisions (how hard should the math ceiling really go, given a strong math background; which General Knowledge topics matter most; should reading passages be short or long), a short set of multiple-choice-style clarifying questions was asked *before* doing the work — rather than guessing and potentially redoing a large chunk of work later. This is a good general strategy for any big, ambiguous task: a two-minute clarifying conversation up front can save hours of rework.

Phase 8 — Scaling Up Into a Real Exam: Competition Math, Smarter Vocabulary, a Coach, and a Real Clock

This was the largest single round of changes, so it's broken into its own pieces.

8a. Competition-Grade Mathematics

The ask: Use a real mathematics-competition document as a style guide (a paper from the Mathematical Association of Nigeria, Enugu chapter) to raise the hardest math tier to genuine competition standard.

An important honesty moment: The actual content of that document wasn't available to work from directly — only a memory that it had existed in an earlier conversation, not its actual questions. Rather than quietly guessing and pretending to match a document that couldn't actually be seen, this limitation was stated openly, and the hardest math tier was instead built using well-known genuine competition-math conventions: functional equations (relationships like " $f(x+y) = f(x) + f(y)$ "), simplifying surds (expressions with square roots), circle geometry theorems, number theory (like finding the remainder when a huge number is divided by a small one, using patterns instead of brute-force calculation), and classic "pigeonhole principle" logic puzzles. Every one of these formulas was, again, tested independently before being trusted — because competition math is exactly the kind of content where a subtle formula mistake is easy to miss.

8b. Rebuilding English Around Real Vocabulary Skill

The ask: Push English harder by promoting each existing tier up a level (what used to be the "Hard" questions became the new "Easy" tier, and so on), and specifically fix how synonyms and antonyms are tested — instead of asking "what's a synonym for X?" (which just tests memorized dictionary definitions), test whether someone can pick the right word *in context*, based on tone and usage — much closer to how vocabulary actually gets used in real writing and real exams.

The strategy: Every synonym/antonym question was redesigned around a sentence, where the challenge isn't "do you know this word" but "can you tell which of these four similar-but-not-identical words actually fits the meaning and tone of this specific sentence." That's a meaningfully harder and more useful skill than flashcard-style memorization.

8c. A Personal Coach: The Recommendation System

The ask: After finishing the test, don't just show a score — tell the person specifically *where* they should focus their studying.

The strategy: Every single question, no matter the subject, was tagged behind the scenes with the specific skill it's testing — for example, "Ratio & Proportion," "Reading Comprehension – Inference," "Number Theory," "Contextual Vocabulary." After the test ends, the app looks at performance broken down by these skill tags (not just by subject), finds the weakest ones, and produces a plain-English recommendation — something like: "You struggled most with Contextual Vocabulary — focus your next study session on how word meaning shifts with tone and sentence context," rather than just "You got 62%." This turns the app from a scorekeeper into something closer to a tutor.

8d. One Clock for the Whole Exam, Not One Per Question

The ask: Replace the per-question countdown timer with a single 45-minute countdown for the *entire* test.

The strategy: This is actually closer to how most real exams work — you manage your own time across the whole paper rather than being force-marched question by question. The single big clock runs continuously from the moment the test starts; if it reaches zero at any point, the test ends immediately wherever you are, and whatever you've completed gets scored as-is. This also better trains a real, transferable skill: personal time budgeting across an entire exam, not just per-question reflexes.

8e. Making Sure You Never See the Same Question Twice

The ask: If someone takes the test multiple times to practice, they shouldn't keep seeing identical questions or identical answer choices.

The strategy: The app needed a memory of what it has already shown a particular person, so it can avoid repeating those exact items on future attempts — and only "reset" and reuse older questions once the entire pool has genuinely been exhausted. For the endlessly-generated math questions, this matters less (since fresh random numbers rarely repeat exactly), but the fixed English/General Knowledge question banks needed this tracking specifically, along with continuing to shuffle the order that multiple-choice options appear in every time.

8f. Making It Feel Like a Real Android App

The ask: Package the whole thing so it can be used like a proper app on an Android phone.

The strategy, and an honest limitation: Building a true installable Android app (the kind you'd download from an app store) requires specialized software-building tools that aren't available in this environment — so that specific outcome (a literal `.apk` file) isn't something that could be produced directly. What *can* be built instead is a "web app" version: a single self-contained file that, when opened in a phone's browser, can be added to the home screen and then behaves almost exactly like a normal app — full screen, its own icon, and (critically) it still works with no internet connection at all, because everything it needs is bundled inside that one file rather than fetched from a server.

Phase 9 — Reverse-Engineering the Psychology of the Real Exam

The ask: Step back from building anything and instead reason abstractly: given that the real exam is 45 questions in 45 minutes with an 85% pass mark, and the committee explicitly wants to screen out as many people as possible, what does that imply about how the *actual* real questions are probably designed — without assuming anything from the earlier uploaded guide document?

The reasoning, explained simply: An 85% cutoff only makes sense as a real filter if the exam is deliberately built with two very different kinds of questions mixed together:

- A large majority of fair, reasonable questions that any well-prepared student can get right — these exist so the test still feels legitimate and isn't pure luck.
- A smaller handful of deliberately tricky questions, engineered to be missed even by strong, well-prepared candidates — these are the questions doing almost all of the actual "screening" work, because if everyone could clear every question, an 85% cutoff wouldn't filter anybody out.

Because there's only about one minute per question on average, those tricky questions can't rely on being *long* or *heavily calculation-based* — there's no time for that. Instead, real screening exams like this tend to hide their difficulty inside apparent *simplicity*: a sneaky "except" or "NOT" hidden in the question wording, a plausible-looking wrong answer that would be correct for a slightly different question, a word problem with one extra irrelevant number thrown in to catch people who skim instead of read carefully, or a math shortcut that's obvious if you spot the pattern and painfully slow if you don't.

Why this insight matters going forward: It reframes what "hard" should actually mean for the next round of practice content. Making questions "5x harder" in the sense of heavier math or denser vocabulary is a different (and less accurate) target than making questions *trickier* in the specific ways a real timed screening exam would be — short, deceptively simple-looking, and punishing to anyone who reads too fast or pattern-matches without checking the details.

How All of This Comes Together

If you laid the entire project out as a single instruction to give someone from scratch, it would read like this:

Build an offline, installable, phone-friendly practice exam for the DCF scholarship aptitude test. It must cover exactly three subjects — English, Mathematics, and General Knowledge — scored from 0 to 600 with a pass mark of 510 (85%), across three difficulty tiers (Easy/Hard/Difficult worth 5/10/15 points each) that unlock progressively based on performance. The test totals 60 questions and is governed by a single 45-minute countdown for the whole session, not per question. Mathematics must never require a calculator, but should be written to SAT Level 1–2 standard for its easier tiers and genuine mathematics-competition standard (functional equations, number theory, geometry theorems, combinatorics) for its hardest tier. English should test grammar, analogies, and reading comprehension at an SAT/WAEC/JAMB/NECO blended standard, with synonyms and antonyms tested through contextual sentence usage rather than direct definition-matching. General Knowledge should span science, technology, history, geography, and culture rather than narrowly focusing on current affairs. No question or answer set should repeat across multiple attempts by the same person until the full question bank has genuinely been exhausted. After each attempt, the candidate should receive a specific, skill-by-skill breakdown of where they underperformed, with concrete recommendations for what to study next — not just a raw score. Every generated question, especially in Mathematics, must be independently verified for correctness before being trusted, since formula-based question generation can silently produce wrong answers if not checked. Finally, because the real

exam appears deliberately designed to screen out most applicants despite a demanding 85% cutoff, a meaningful portion of the hardest-tier content should specifically train *precision under time pressure and trap-detection* — deceptively simple-looking questions with hidden qualifiers or subtle wrong-answer traps — rather than relying purely on raw content difficulty to create challenge.

What's Still Open

A few things remain genuinely unresolved and would need your input (or your materials) to finish properly:

1. **The actual MAN Enugu State chapter document** hasn't been seen directly — only referenced from memory. If you re-share it, the competition-math tier can be recalibrated to match its real style precisely instead of general competition-math conventions.
2. **The real DCF exam's actual questions** have never been seen — everything above about "trap questions" and "hidden difficulty" is informed reasoning about what a rational exam committee would likely do, not confirmed fact.
3. **A true installable Android .apk** was never produced — only the offline, add-to-home-screen web app version, which behaves like an app but isn't a store-installable package.

If you want, the next concrete step would be actually building the Phase 9 "trap-question" content into the practice app, since that's the one strategic insight from this whole project that hasn't been turned into real practice material yet.
